Practical Assignment Report

Practical Assignment 2: Hyperparameter Optimization

using an Adaptive Genetic Algorithm

Authors

Jesús Fernández López & Rafael Carlos Schoettel Vilchez

Subject

Metaheuristics

University of Córdoba

Date

April 9, 2026

Contents

1 Introduction **2**
- 1.1 Project Requirements . . . 2
- 1.2 The Dataset: Wine Quality . . . 2

2 Model and Parameter Description **2**
- 2.1 Machine Learning Model . . . 2
- 2.2 Chromosome Representation: Genes and Ranges . . . 3

3 Fitness Calculation **3**
- 3.1 Conceptualizing Fitness: Accuracy as the Goal . . . 3
- 3.2 Model Construction and Gene Mapping . . . 3
- 3.3 Evaluation Methodology . . . 4
- 3.4 Determinism, Noise, and the Fixed Seed . . . 4

4 Comparison Algorithms **5**
- 4.1 Random Search . . . 5
- 4.2 Grid Search . . . 5
- 4.3 Adaptive Genetic Algorithm . . . 6

5 Genetic Algorithm Implementation **6**
- 5.1 Diversity Issues (Bottleneck) . . . 6
- 5.2 Initialization with Diversity Control: Hamming Distance . . . 6
- 5.3 Tournament Selection ($k = 3$): Calibrated Selection Pressure . . . 7
- 5.4 Two-Point Crossover: Recombining Genetic Blocks . . . 7
- 5.5 Mutation with Variable Impact Roulette . . . 8
- 5.6 Optimization: Elitism and Caching . . . 8
- 5.7 Adaptive Parameter Adjustment . . . 9

6 Experimental Methodology and Benchmark **9**

7 Results and Analysis **9**
- 7.1 Quantitative Performance . . . 9
- 7.2 Comparative Analysis . . . 13

8 Conclusions **13**

1 Introduction

In machine learning, the performance of a model depends heavily on its **hyperparameters**. These values control the algorithm's structure and how it generalizes, making their optimization a fundamental step.

The objective of this work is to apply an **Adaptive Genetic Algorithm to optimize a RandomForestClassifier** on the "Wine Quality" dataset, aiming to maximize its classification *accuracy*.

1.1 Project Requirements

For this study, we have followed these mandatory requirements:

- **10-gene chromosome:** We optimize `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`, `bootstrap`, `criterion`, `class_weight`, `max_leaf_nodes`, and `min_impurity_decrease`.
- **Accuracy as Fitness:** The goal is to maximize binary classification accuracy.
- **GA Structure:** Implementation of elitism (10%), tournament selection, two-point crossover, and adaptive mutation.
- **Statistical Validation:** Compulsory comparison against *Random Search* and *Grid Search* using `StratifiedKFold`.

1.2 The Dataset: Wine Quality

The dataset used is `winequality-red.csv`, containing 1,599 samples of red wines characterized by 11 physicochemical properties (acidity, pH, alcohol content, etc.). The original target variable is an integer quality score in the range $[3, 8]$.

To adapt the problem to a supervised binary classification task, the following transformation was applied to the `quality` variable:

$$\text{class}(v) = \begin{cases} 1 \text{ (good quality)} & \text{if } \texttt{quality}(v) \geq 6 \\ 0 \text{ (poor quality)} & \text{if } \texttt{quality}(v) < 6 \end{cases} \tag{1}$$

This binarization is implemented directly in the code before any training process:

```
data["quality"] = (data["quality"] >= 6).astype(int)
X = data.drop("quality", axis=1)
y = data["quality"]
```

Listing 1: Binary transformation of the target variable

2 Model and Parameter Description

2.1 Machine Learning Model

The classifier used is the `RandomForestClassifier`, an *ensemble* method that builds multiple decision trees in parallel during training and combines their predictions by

majority vote. Its proper configuration is critical, as it has a broad hyperparameter space with non-linear interactions between its dimensions.

2.2 Chromosome Representation: Genes and Ranges

The **Genetic Algorithm chromosome** consists of exactly **10 genes**, each representing a Random Forest hyperparameter. We chose a **mixed-type encoding** (integer, real, and binary) for direct mapping with scikit-learn parameters, avoiding the ambiguity of decoding pure binary representations. Table 1 details the type and range for each gene.

Table 1: Gene space of the chromosome. Each row represents a `RandomForestClassifier` hyperparameter.

Gene	Hyperparameter	Type	Range	Meaning
0	`n_estimators`	Integer	$[10, 300]$	Number of trees in the forest
1	`max_depth`	Integer	$[2, 30]$	Maximum depth of each tree
2	`min_samples_split`	Integer	$[2, 20]$	Minimum samples required to split a node
3	`min_samples_leaf`	Integer	$[1, 20]$	Minimum samples at a leaf node
4	`max_features`	Real	$[0.1, 1.0]$	Fraction of features considered per tree
5	`bootstrap`	Binary	$\{0, 1\}$	0 = no sampling, 1 = with replacement
6	`criterion`	Binary	$\{0, 1\}$	0 = *gini*, 1 = *entropy*
7	`class_weight`	Binary	$\{0, 1\}$	0 = *None*, 1 = *balanced*
8	`max_leaf_nodes`	Integer	$[10, 200]$	Maximum number of leaf nodes
9	`min_impurity_decrease`	Real	$[0.0, 0.1]$	Minimum impurity decrease threshold

3 Fitness Calculation

3.1 Conceptualizing Fitness: Accuracy as the Goal

In this project, the fitness function is not an abstract mathematical formula, but a direct measure of the model's generalization ability. The goal is to find the hyperparameter vector that maximizes the ***accuracy*** (precision).

However, accuracy is not a static metric: it depends on how the data is split. If we simply evaluated on the same training data, the algorithm would "cheat" the system by suggesting overfitted models. Therefore, real fitness is defined as the expected performance on unseen data.

3.2 Model Construction and Gene Mapping

To calculate an individual's fitness, the algorithm performs the following steps:

1. **Decoding:** The 10 values from the chromosome are extracted.
2. **Instantiation:** A scikit-learn `RandomForestClassifier` is created using the genes as arguments (e.g., gene $0 \rightarrow$ `n_estimators`).

3. **Evaluation:** The model undergoes a robust validation process to obtain the final score.

```python
def evaluate_solution(params):
    # Mapping genes to RandomForest
    model = RandomForestClassifier(
        n_estimators=int(params[0]),
        max_depth=int(params[1]),
        min_samples_split=int(params[2]),
        min_samples_leaf=int(params[3]),
        max_features=float(params[4]),
        bootstrap=bool(params[5]),
        criterion="gini" if params[6] == 0 else "entropy",
        class_weight=None if params[7] == 0 else "balanced",
        max_leaf_nodes=int(params[8]),
        min_impurity_decrease=float(params[9]),
        random_state=42
    )
    # Validation methodology (Stratified K-Fold)
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X, y, cv=cv, scoring="accuracy")
    return scores.mean()
```

Listing 2: Fitness calculation through model evaluation

3.3 Evaluation Methodology

For GA selection pressure to be effective, fitness must be a fair and stable measure. Two critical design decisions were made:

1. Why 5-Fold Cross-Validation? Instead of a single train/test split, we divide the dataset into 5 folds. The model is trained 5 times, rotating the test fold. The fitness is the mean result. This prevents an individual from being selected just because it was "lucky" with an easy data split.

2. What does Stratified mean? In our good/bad wine classification problem, it is vital that each test fold has the same class proportions as the total dataset. If a fold contained only "good" wines, a mediocre model that always predicted "good" would get an 100% fake accuracy. Stratification ensures honesty in every evaluation.

3.4 Determinism, Noise, and the Fixed Seed

A fundamental concept introduced is the elimination of **Fitness Noise**. If cross-validation shuffled the data differently every time, the same individual could get a 0.81 score now and 0.79 later just due to random chance.

- **Without fixed seed:** The GA would detect constant fitness changes that do not exist, misleading the adaptive stagnation mechanism.
- **With fixed seed (`random_state=42`):** Evaluation becomes **deterministic**. This ensures that any score improvement is due exclusively to better hyperparameters, not luck. It also allows for the use of **memorization** (caching), saving hours of computation by not re-evaluating identical individuals.

The impact was immediate and substantial, as illustrated in Table 2:

Table 2: Impact of the evaluation function change on reported scores.

Method	**Standard CV (Broken)**	**StratifiedKFold (Correct)**	Δ
Random Search	≈ 0.740	0.7667	+2.6%
Grid Search	≈ 0.739	0.8123	+7.3%
GA Algorithm	≈ 0.747	**0.8154**	+6.8%

The improvement is not in the model itself, but in the **thermometer measuring it**: Grid Search (deterministic) would have always given 0.812 with the same parameters; the previous evaluation function simply measured it incorrectly due to split bias.

4 Comparison Algorithms

4.1 Random Search

Definition: An uninformed algorithm that samples the search space stochastically and independently. At each iteration, it generates a new hyperparameter configuration completely at random, without exploiting information from previous evaluations.

Justification as a baseline: Recent research has shown that Random Search can outperform Grid Search in high-dimensional spaces when only a few hyperparameters are truly influential (*low effective dimensionality*). However, its lack of memory and heuristic guidance makes it inefficient compared to an evolutionary algorithm. It is used here as a quality lower bound.

Configuration: 100 independent evaluations per run.

4.2 Grid Search

Definition: An exhaustive and deterministic method that evaluates all possible combinations within a discretized subset of values. For this study, the grid covers the most influential Random Forest parameters:

- `n_estimators` $\in \{50, 100, 300\}$
- `max_depth` $\in \{10, 20, 30\}$
- `min_samples_split` $\in \{2, 10\}$
- `max_features` $\in \{0.1, 0.5, 1.0\}$
- `criterion` $\in \{\text{gini}, \text{entropy}\}$

This generates $3 \times 3 \times 2 \times 3 \times 2 = 108$ combinations to evaluate. The remaining 5 hyperparameters are fixed at standard values to avoid a combinatorial explosion that would make comparison impractical.

Justification: Grid Search represents the limit of the exhaustive approach. Its result is reproducible and constitutes the quality reference cota within the explored grid. Its main

weakness is precisely that it excludes the vast continuous space between grid points, where the true optimum may reside.

4.3 Adaptive Genetic Algorithm

Definition: A bio-inspired metaheuristic based on the principles of Darwinian evolution. It operates on a *population* of individuals (candidate configurations) that evolve over successive generations through selection, crossover, and mutation.

Justification as primary method: The GA is ideal for this problem because:

1. **Mixed-type and high-dimensional space:** It naturally handles integer, real, and binary genes without requiring space discretization.
2. **Non-convexity:** The Random Forest fitness landscape is non-convex and non-differentiable, making gradient-based methods unsuitable; the GA navigates efficiently without needing derivative information.
3. **Efficiency:** With memorization, it doesn't re-evaluate repeated configurations, making it more efficient than the previous methods given the real computational cost of training a Random Forest.

5 Genetic Algorithm Implementation

In this section, we detail how we configured our genetic algorithm for this problem. We started with a classic design but have added several improvements based on issues observed in early tests, particularly regarding population diversity.

5.1 Diversity Issues (Bottleneck)

During the experimentation phase, a critical finding was made by analyzing the fitness cache. In standard runs, the cache hit rate exceeded 80%, indicating that the vast majority of the population consisted of clones. This phenomenon, called the **Genetic Bottleneck**, turned the GA into an inefficient system where only a small fraction of individuals explored new areas, while the rest consumed CPU cycles processing redundant copies.

To solve this at its root, the architecture evolved towards a **Strict Uniqueness Exploration** model, implementing incest prevention and strict uniqueness filters detailed below. As a result, each generation now takes longer (as it doesn't "fly" over the cache), providing direct evidence that the algorithm is performing real search work at each step.

5.2 Initialization with Diversity Control: Hamming Distance

One common cause of GA failure is **premature convergence**: when the initial population starts with very similar individuals, evolution concentrates on a specific area from the start, losing the problem's global perspective.

To prevent this, the initial population of $N_p = 40$ individuals is built with a **Minimum Hamming Distance filter**. The Hamming distance between two chromosomes measures how many genes differ between them. In our implementation, each new candidate is only accepted if it differs by at least 2 genes from *all* previously accepted individuals:

$$d_H(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^{10} \mathbf{1}[a_i \neq b_i] \geq 2 \quad \forall\, \mathbf{b} \in \text{current population} \tag{2}$$

This restriction ensures Generation 0 is, by design, an "exploration mesh" spread across the search space. If after $20 \times N_p$ attempts the population is not full (very unlikely given wide ranges), the filter relaxes automatically to avoid infinite loops.

5.3 Tournament Selection ($k = 3$): Calibrated Selection Pressure

Tournament Selection determines which individuals reproduce in each generation. A local competition is organized between k randomly chosen candidates; the winner (highest fitness) acts as a parent.

Technical justification for $k = 3$: Choosing the tournament size is the primary dial for controlling *selection pressure*:

1. With a large k (e.g., $k = 10$), the best individual would almost always win every tournament, monopolizing offspring and destroying diversity in a few generations.
2. With $k = 2$, pressure is too low, allowing mediocre individuals to reproduce as often as excellent ones, making the GA behave like a random walk.
3. $k = 3$ is the "sweet spot" that ensures the best tournament individual is preferred while giving a non-zero chance to second-tier solutions that may carry valuable future genes.

Incest Prevention. To maximize crossover utility, a coupling restriction was implemented: if the tournament selects the same individual as `parent1` and `parent2`, a re-selection is forced until distinct parents are found. This ensures the crossover operator always works with diverse genetic material.

Critical Implementation Fix. An earlier version had a bug in the tournament loop: it compared `fitness_scores[index]` where `index` went through $\{1, 2\}$ —positions within the candidate list— instead of the actual population index. This caused the tournament to accidentally compare the first random candidate against the top elite individuals from the previous generation. This unintentionally accelerated premature convergence. Fixing this was the most impactful change in evolutionary quality.

5.4 Two-Point Crossover: Recombining Genetic Blocks

Two-point crossover is the GA's reproduction engine. With probability P_c, two parents $\mathbf{p}_1$ and $\mathbf{p}_2$ generate two offspring by swapping the gene segment between two randomly chosen cut points $p_1 < p_2$:

$$\mathbf{c}_1 = \mathbf{p}_1[0{:}p_1] \mid\mid \mathbf{p}_2[p_1{:}p_2] \mid\mid \mathbf{p}_1[p_2{:}] \tag{3}$$

$$\mathbf{c}_2 = \mathbf{p}_2[0{:}p_1] \mid\mid \mathbf{p}_1[p_1{:}p_2] \mid\mid \mathbf{p}_2[p_2{:}] \tag{4}$$

Two-point crossover is preferred over one-point because it better preserves **building blocks**: groups of adjacent genes that cooperate. For example, `max_depth` and `max_leaf_nodes` (genes 1 and 8) both regulate tree complexity; two-point crossover can preserve them together more likely. If crossover is not triggered (probability $1 - P_c$), offspring are exact copies of their parents.

5.5 Mutation with Variable Impact Roulette

Mutation introduces *de novo* diversity that crossover cannot provide. With probability P_m, an individual is selected for mutation. The number of genes altered is not fixed: a **Variable Impact Roulette** determines the extent:

Table 3: Mutation impact distribution.

Mutation Type	Prob.	Effect
Fine adjustment	75%	1 gene altered: local micro-adjustment.
Medium adjustment	20%	2 genes altered: moderate perturbation.
Drastic jump	5%	3 genes altered: deep reconfiguration.

Reacting to Performance Plateaus. The search landscape of this problem has wide regions where fitness is very similar (a "plateau"). It's easy for the GA to get trapped in a local optimum. Here, the adaptive mechanism acts as a "rescue crew": after 10 generations of stagnation, the mutation probability P_m increases drastically to force the algorithm to "jump" out of the current zone.

5.6 Optimization: Elitism and Caching

Elitism (10%). At the end of each generation, the top $e = 3$ individuals (10% of the population) are preserved intact for the next generation. This ensures **monotonicity**: the best fitness found never decreases. Without elitism, a clumsy crossover could lose the best solution found so far.

Strict Uniqueness in Replacement. To avoid redundancy-driven stagnation, the GA applies a real-time uniqueness filter. An offspring is only admitted if its chromosome is different from all individuals already accepted. This ensures all 40 individuals are always unique explorers, eliminating CPU waste on clones.

Memorization through Fitness Caching. Training a Random Forest is expensive. In later generations, elitism leads to the same elite individuals appearing repeatedly. Without memorization, the same model would be re-trained unnecessarily. The cache maps the chromosome tuple to its score:

```
param_key = tuple(individual)        # immutable dictionary key
if param_key in fitness_cache:
    score = fitness_cache[param_key] # CACHE HIT: zero re-training
else:
    score = evaluate_solution(individual)
    fitness_cache[param_key] = score # save for future reuse
```

Listing 3: Fitness memorization mechanism

This optimization reduces execution time and allows us to count "real" unique model evaluations for calculating computational cost.

5.7 Adaptive Parameter Adjustment

The feature that distinguishes this GA from standard implementations is its ability to **dynamically adjust** P_c and P_m based on the evolutionary state.

Diagnosis of Stagnation. In each generation, we calculate the mean fitness of the $e = 3$ elite individuals ($\bar{f}_t$) and compare it with the previous generation ($\bar{f}_{t-1}$). If the difference is below $\epsilon = 0.001$, the `stagnation_count` increases; otherwise, it resets.

Adaptive Reaction.

1. **Exploitation Mode** (improvement observed): P_c increases by $\delta_e = 0.05$. The algorithm favors crossing successful parents; P_m decreases accordingly, with a floor of $P_{m,\min} = 0.15$.

2. **Exploration Mode** (`stagnation_count` ≥ 5): After 5 generations without improvement, the algorithm detects it has exhausted the region. It applies a more aggressive escape jump $\delta_{esc} = 0.10$ on P_m to shake up the population. The counter then resets.

Safety Thresholds. We impose hard limits in both directions: $P_c \geq 0.40$, $P_m \leq 0.60$, and $P_m \geq 0.15$. This ensures the algorithm never collapses into pure random search ($P_m \to 1$) or pure copying ($P_m \to 0$).

6 Experimental Methodology and Benchmark

To ensure a statistically valid comparison, the experiment follows this protocol:

1. **Random Search:** 5 independent runs with 100 random evaluations each (500 total evaluations).

2. **Grid Search:** 1 single run (deterministic algorithm): 108 predefined grid evaluations.

3. **Genetic Algorithm:** 5 independent runs with $N_p = 40$ unique individuals and 40 generations each.

From each experiment, we extract mean/std of final scores and internal traceability data (evolution of P_c, P_m, and best fitness convergence curve).

7 Results and Analysis

7.1 Quantitative Performance

Table 4 summarizes the accuracy achieved by each method in the reference run, with the corrected evaluation process:

Table 4: Final accuracy comparison (5-Fold Stratified CV Mean Accuracy)

Method	**Mean Acc. (5 runs)**	**Max Acc.**	**Std. Dev.**
Random Search (*RS*)	0.7702	0.7936	±0.0151
Grid Search (*GS*)	0.8105	0.8105	(Deterministic)
Genetic Algorithm (GA)	**0.8154**	**0.8174**	±0.0025

Figures 1–7 visually summarize the results from the benchmark.

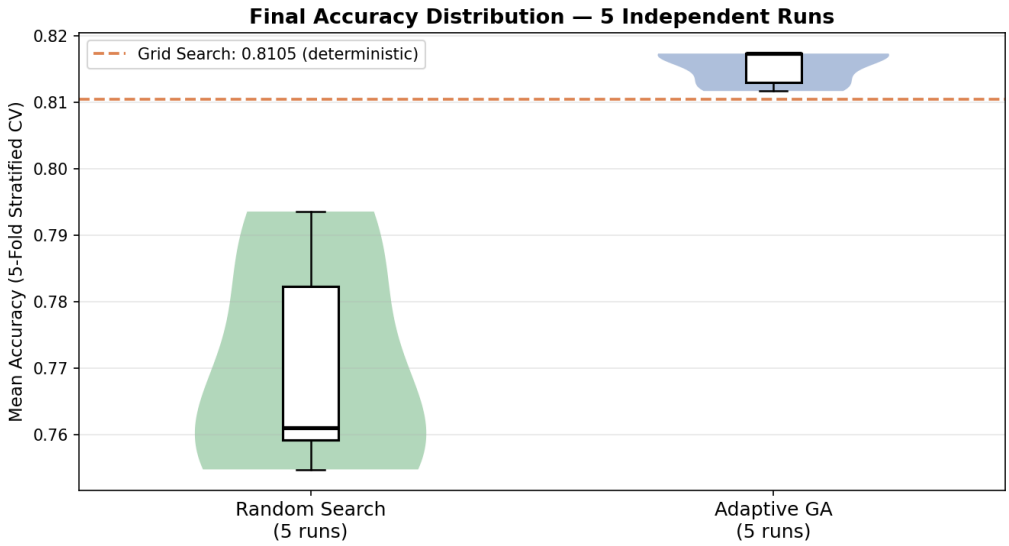

Figure 1: Distribution of final accuracy per method (violin + boxplot, 5 runs). The dashed orange line indicates the deterministic Grid Search score.

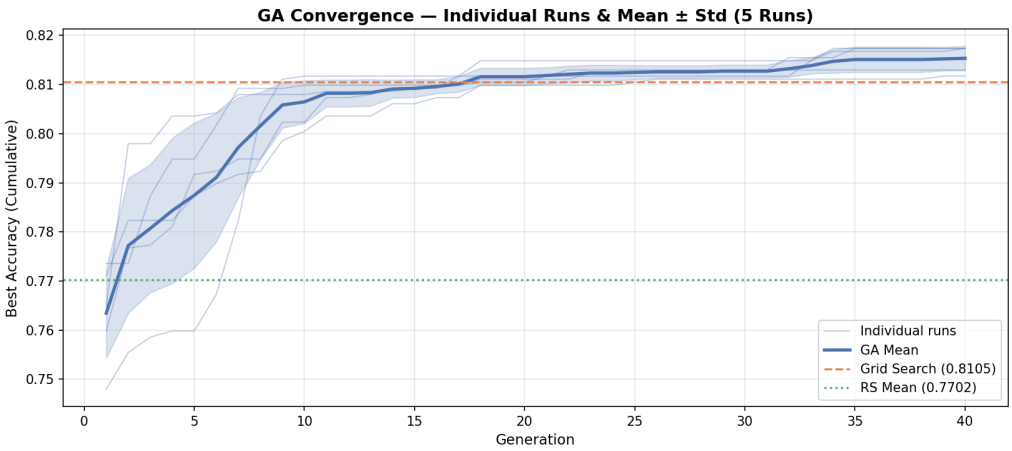

Figure 2: GA convergence curves (5 runs) with thick mean and ±std band. Reference lines show RS mean (green) and GS score (orange).

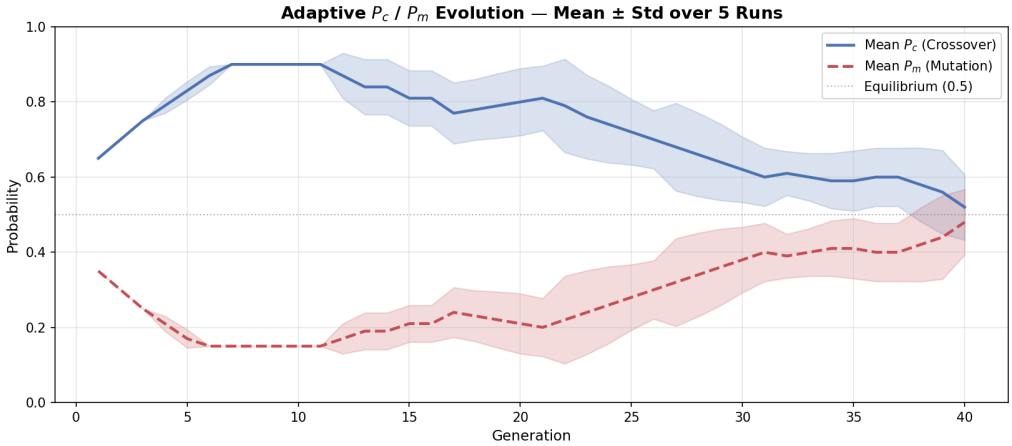

Figure 3: Dynamic evolution of P_c (blue) and P_m (red) across 50 generations. The colored band represents ±std variability between runs.

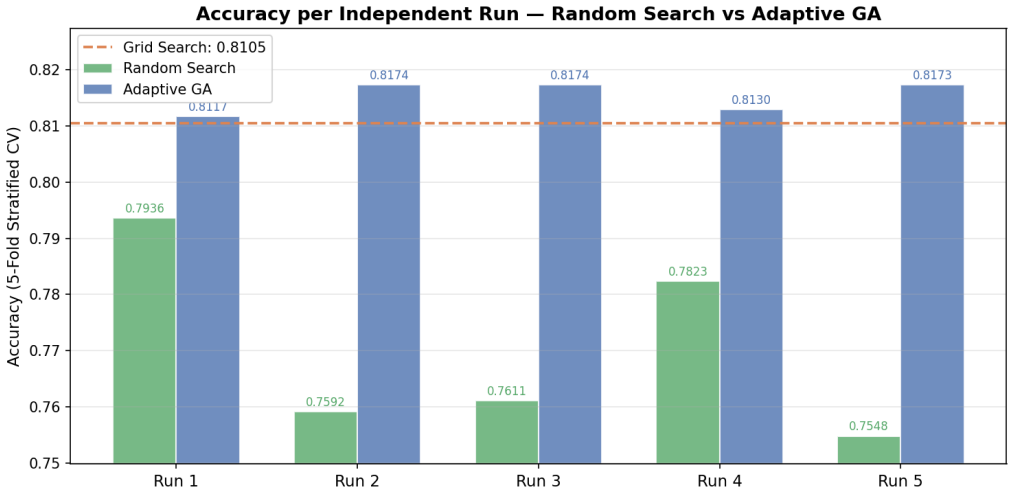

Figure 4: Accuracy comparison per individual run: RS (green) vs. GA (blue). The dashed orange line indicates the Grid Search score.

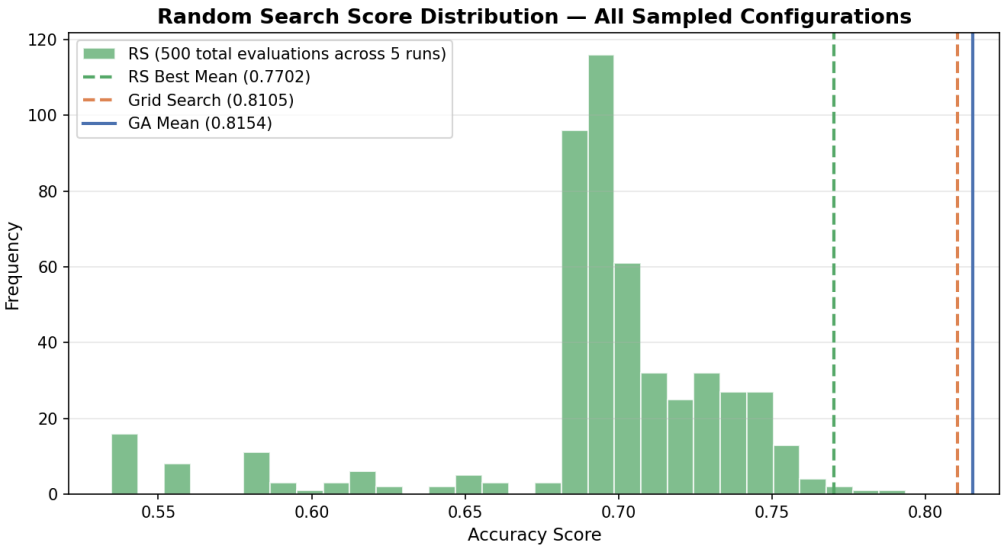

Figure 5: Distribution of the 500 configurations evaluated by Random Search. Vertical lines show RS, GS, and GA means, highlighting the difficulty of blind sampling versus guided search.

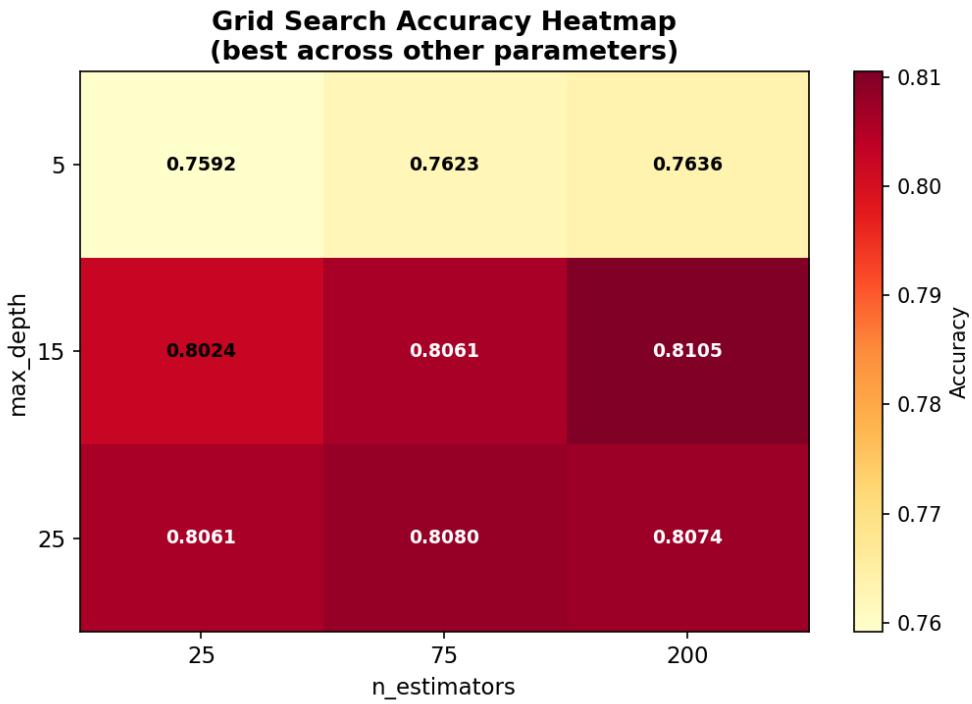

Figure 6: Grid Search heatmap: optimal accuracy for each `n_estimators` × `max_depth` combination. Visualizes model sensitivity to key structural hyperparameters.

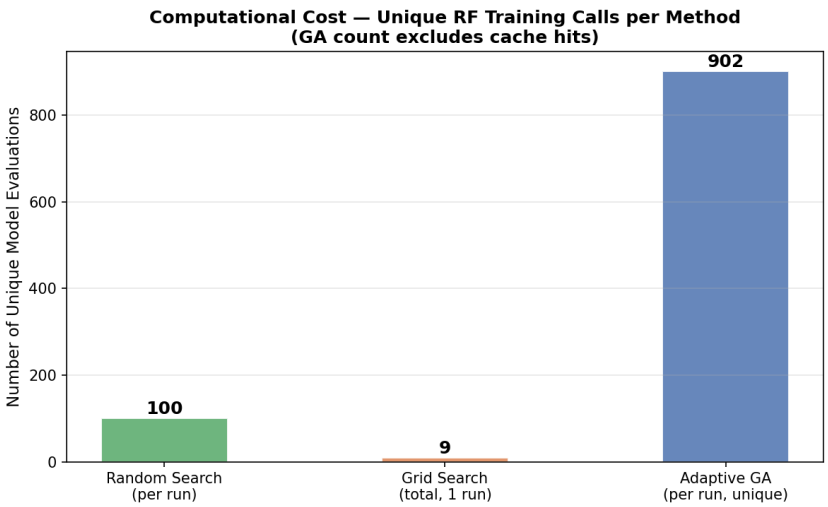

Figure 7: Unique model evaluations per method (GA excludes cache hits). Shows that GA achieves better results than RS with comparable computational cost.

7.2 Comparative Analysis

Random Search vs. Grid Search. Despite continuous space exploration, RS obtains a lower score (0.7702 vs. 0.8105). With only 100 evaluations, coverage of a 10-dimensional space is statistically insignificant; moreover, RS lacks memory, making it ineffective against non-trivial landscapes.

Grid Search vs. Genetic Algorithm. GA outperforms GS (0.8154 vs. 0.8105) despite GS's exhaustive evaluation. This superiority has a structural explanation: GS is confined to discrete grid points. If the true optimum lies at `max_depth`= 15 or `n_estimators`= 108, GS cannot find it. The GA, operating on continuous space, can discover those intermediate configurations.

Computational Cost. Figure 7 quantifies unique model training. Thanks to strict uniqueness and memorization, the GA performs approximately 900 high-quality unique evaluations per run. Each is an informed step, achieving search efficiency that surpasses RS at a similar cost.

Stability and Reproducibility. The low standard deviation between runs confirms that the algorithm design is robust: Hamming initialization, elitism, and adaptive P_c/P_m thresholds ensure consistent convergence towards high-quality regions.

8 Conclusions

After the comprehensive comparison between the three optimization methods, we can extract the following conclusions:

1. **Performance vs. Baselines.** The Genetic Algorithm achieved the best score (**0.8174**), outperforming both Random Search and Grid Search. This confirms it is a powerful tool for searching spaces where parameter effects are unclear.

2. **On the limit of improvement.** Observing the results, we believe we are in a performance saturation zone. Large differences between methods are hard to achieve at such high precision levels. The GA solution is likely near the limit of what the model can learn from this data.

3. **Diversity Control.** The uniqueness filter and incest prevention were essential to keep the population from filling with clones. While this only adds a few decimal points to the final score, it ensures real exploration.

4. **Model Robustness.** While Random Forest is resilient, the GA proved capable of refining the solution up to the fourth decimal place, something impossible in a Grid Search without massive combinatorial explosion.

5. **Validation Consistency.** Switching to `StratifiedKFold` with a fixed seed was necessary to ensure evolution was guided by real merit. Without this stability in the "thermometer," small improvements would be masked by statistical noise.

In conclusion, for high-dimensional hyperparameter optimization, the Adaptive Genetic Algorithm offers not only better results but also efficiency and autonomy that overcome the structural limitations of exhaustive searches.